

LAB 9a

JAVASCRIPT 2: DOM AND EVENTS

What You Will Learn

- Accessing and modifying DOM HTML elements using JavaScript
- Creating event listeners to react to events
- Tools and tricks to help you develop JavaScript

Approximate Time

The exercises in this lab should take approximately 120 minutes to complete.

Fundamentals of Web Development, 3rd Ed

Randy Connolly and Ricardo Hoar

Textbook by Pearson
<http://www.funwebdev.com>

Date Last Revised: August 25, 2025

Revised: Split lab09 into two labs (lab09a and lab09b).
Revised: Moved old exercises 9.9-9.12 into lab09b.
Revised: Added new exercises: 9a.7, 9a.8, and 9a.14.
Revised: Added new Test Your Knowledges: #3, #4 and #5.
Revised: Moved old Test Your Knowledge #4 to Lab9b.

PREPARING DIRECTORIES

- 1 The starting `lab09a` folder has been provided for you (within the zip folder downloaded from Gumroad).

Note: these labs use the convention of blue background text to indicate filenames or folder names and bold red for content to be typed in by the student.

THE DOCUMENT OBJECT MODEL (DOM)

In the previous lab, you used the `document.write()` function as a way to manipulate markup using JavaScript. While fine from a learning JavaScript perspective, it is generally not how one typically uses JavaScript. In the first part of this lab, you will learn how to use the Document Object Model (DOM) with JavaScript as a “better” way of manipulating content.

EXERCISE 9a.1 — THE DOCUMENT OBJECT

- 1 Examine `lab09a-ex01.html` in your browser and then in your editor of choice.
- 2 Display the JavaScript console in your browser and enter the following command in the console:

```
console.dir(document)
```

This provides a hierarchical view of the document object, which allows you to examine the different properties and methods within this object.

- 3 Examine some of these properties and methods. Notice that some of these properties are hierarchical themselves (e.g., the `all` property, which is a collection of all the elements in the document).

Working with the DOM means working with the document object.

EXERCISE 9a.2 — BASIC DOM SELECTION

- 1 Examine `lab09a-ex02.html` in your browser. This is the same file except for the addition of a `<script>` element.

Notice that it references a JavaScript file at the end of the markup. The reason why it is after all the markup has to do with ensuring the DOM is loaded. To better understand the rest of the steps in this exercise, you should continually refer back to the markup.

- 2 Edit `js/lab09a-ex02.js` by adding the following and then testing `lab09a-ex02.html` in browser.

```
const msg = document.getElementById("msg");  
msg.value = "this text is generated by JavaScript";
```

This selects the `<textarea>` element and sets its value.

- 3 Add the following code and test.

```
const legend = document.getElementById("title");  
legend.textContent = "Dynamically generated";
```

The `textContent` property allows you to modify the content of an element.

- 4 Replace the two previous lines with the following single line and test:

```
document.getElementById("title").textContent =  
    "Dynamically generated";
```

JavaScript developers often try to reduce the number of variables that get added into scope by not defining variables if they are not needed.

- 5 Comment out the previous line and add the following code and test.

```
const legend = document.getElementById("title");  
legend.innerHTML = "Dynamic with <span>HTML</span> added";
```

Notice that you can “inject” HTML into the content of an element. As we shall see below, while easy, we often want to take a more complicated route when adding HTML elements to a document in order to ensure new content gets added to the DOM tree.

- 6 Modify the code as follows and test.

```
const legend = document.getElementsByTagName("legend");  
legend[0].innerHTML = "Dynamic with <span>HTML</span> added";
```

Here you are using a different selection method, that returns a list of `<legend>` elements. It doesn't matter if there is only a single `<legend>` element, the `getElementsByTagName` method always returns a `HTMLCollection` element (an array-like object that can be iterated).

- 7 Add the following and test:

```
const labels = document.getElementsByClassName("input-labels");  
for (let lab of labels) {  
    lab.style.backgroundColor = "#FFF0F0";  
}
```

Since `getElementsByClassName` returns a `HTMLCollection`, we can iterate through it. In this example, we programmatically alter the `background-color` CSS property.

- 8 Add the following code and test.

```
document.querySelector("#msg").style.color = "#82898c";
```

The `querySelector` method allows you to select a single element using a CSS selector. Unless you need to support older (pre-2012) browsers, we recommend using `querySelector` and `querySelectorAll` instead of `getElementById`, `getElementsByClassName`, and `getElementsByTagName`.

- 9 Add the following code and test.

```
const inputs = document.querySelectorAll("input[type=text]");
for (let inp of inputs) {
    inp.style.backgroundColor = "#E4F0F5";
}
```

The `querySelectorAll` method allows you to select multiple elements using a CSS selector. It returns a `NodeList`, which is an array-like object that can be iterated.

- 10 Add the following code and test. Examine the error message in the console.

```
// this will generate a runtime error ... why?
inputs.style.color = "red";
```

Since `querySelectorAll` returns a `NodeList`, the `style` property will be undefined.

- 11 Comment out the previous line of code, and then add the following code and test.

```
const notOkay = document.querySelector("#msg000");
alert(notOkay);
```

Notice that if the selector passed to `querySelector` doesn't match any elements, it returns `null`.

TEST YOUR KNOWLEDGE #1

Examine `lab09a-test01.html` and then open `lab09a-test01.js` in your editor. Modify the JavaScript file to implement the following functionality.

- 1 Use `getElementById` to add a border via CSS to the `<ul>` element with the name `"thumb-list"`.
- 2 Use `querySelector` to set the `value` property of the `<textarea>` to the `textContent` of the `<p>` element within the `<fieldset>`.
- 3 Use `querySelectorAll` to add a box shadow to each of the `<img>` elements within the `<ul>` element. The CSS property name is `box-shadow` so the JavaScript DOM property name will be `boxShadow`. To see a sample `box-shadow`, look at the example for the `box` class in `lab09a-ex01.css`. Remember that you will need to use a loop. The result should look similar to that shown in Figure 9.1.

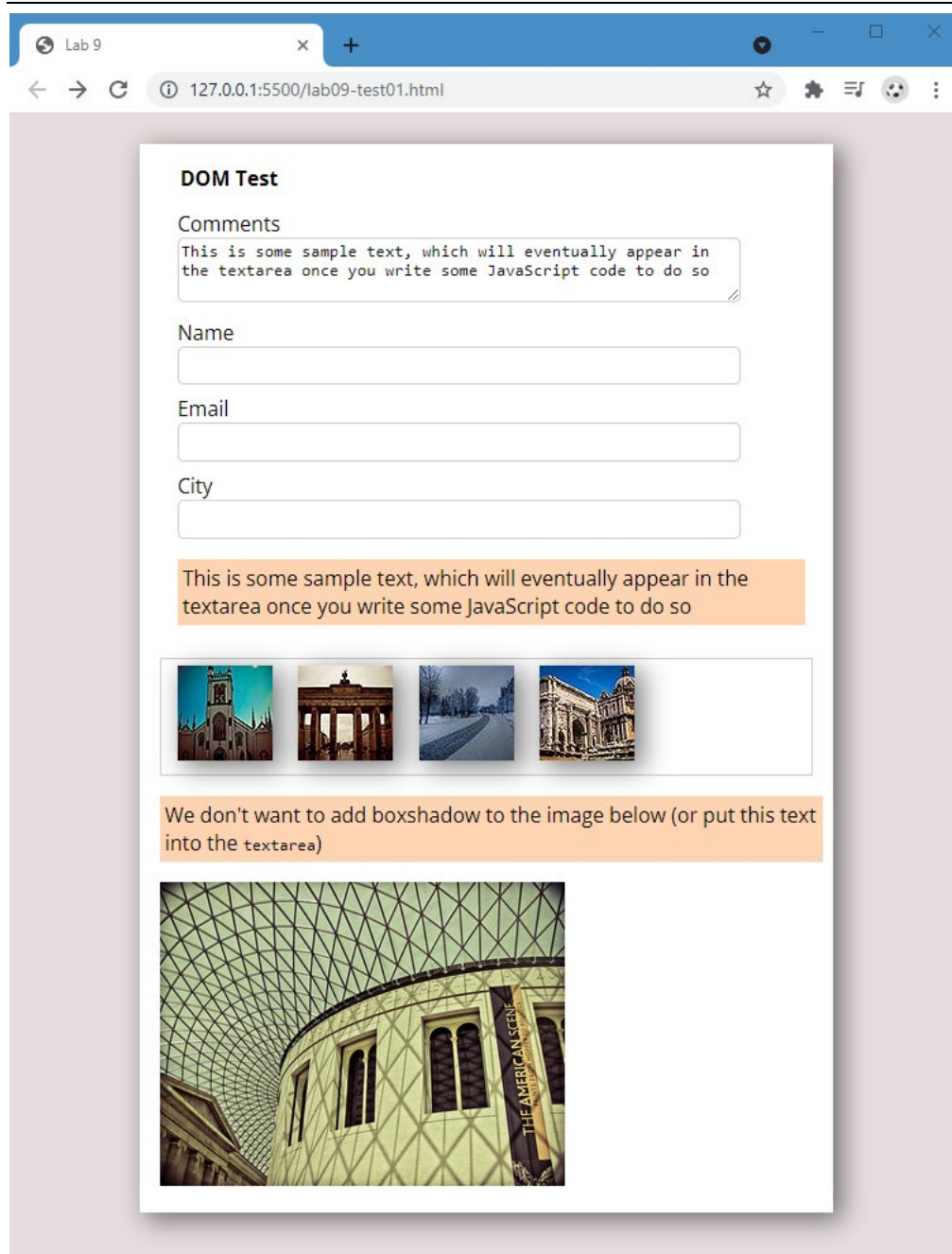


Figure 9.1 – Finished Test Your Knowledge #1

EXERCISE 9a.3 — MODIFYING THE DOM

- 1 Examine `lab09a-ex03.html` in your browser. Open `lab09a-ex03.js`, add the following, and test.

```
// select the <ul>
const list = document.querySelector("#thumb-list");

// create list item <li>
const item = document.createElement("li");

// create <img> element and set its src attribute
const img = document.createElement("img");
img.setAttribute("src", "images/8710320515.jpg");

// nest the <img> in the <li>
item.appendChild(img);

// add the <li> after the existing <li> elements
list.appendChild(item);
```

This illustrates the proper way to dynamically add HTML elements to a document. As you can see, this code adds a new image to your list.

- 2 Comment out the `setAttribute` line in the last step, add the following and test.

```
const img = document.createElement("img");
//img.setAttribute("src", "images/8710320515.jpg");

// some attributes have built-in properties you can use instead
img.src = "images/8710320515.jpg";
```

Which is the best approach for setting attributes? Some prefer using `setAttribute` since it always works (not all attributes have dedicated properties); you will likely encounter both approaches.

- 3 Comment out the last line in the first step, add the following and test.

```
//list.appendChild(item);

// add image instead to the start of the list
list.insertAdjacentElement('afterbegin', item);
```

As you can see, this adds the new image to the beginning of the list.

- 4 Add the following code and test.

```
// the parent of the ul list is the div ... did it work as expected?
const div = list.parentNode;
console.log(div);

// you hope that this selects the h1, but does it?
const h1 = list.previousSibling;
console.log(h1);

// you hope that this selects all the li items, but did it work?
const listItems = list.childNodes;
console.log(listItems);
```

If you examine the console, you will see that two of these didn't work as expected. `h1` is not an element node, but a text node. It has selected the text node containing the line break.

and spaces; white spaces are nodes! Similarly, `listItems` contains both text nodes (white space) and element nodes/

- 5 Comment out the previous code and replace it with the following, then test.

```
const h1 = document.querySelector("main h1");
console.log(h1);

const listItems = document.querySelectorAll("#thumb-list li");
console.log(listItems);
```

Now this works as expected. You are almost always better off making use of the proper descendant selector and `querySelector` or `querySelectorAll` instead of using the sibling and child properties.

- 6 Add the following lines and test.

```
// remove the last <li>
document.querySelector("#thumb-list li:last-child").remove();
```

This deletes the last `<li>` element.

- 7 Comment out the line from the previous step, add the following, and test.

```
//document.querySelector("#thumb-list li:last-child").remove();
// remove all children (i.e., every li within the ul)
while (list.firstChild) {
  list.removeChild(list.firstChild);
}
```

Here is an example where the child properties are useful. Every node element has a `firstChild` property, which will be non-null if it has any child elements.

- 8 Comment out the `while` loop and replace it with the following.

```
list.innerHTML = "";
```

This is a simpler (and faster in terms of execution speed) way to remove all children elements from a node.

EXERCISE 9a.4 — CHANGING CSS CLASSES

- 1 Examine `lab09a-ex04.html` in your browser. In this exercise, you will make the element with the sign-in form appear after a time delay.

- 2 Modify the `<aside>` element by adding the following class and test.

```
<aside class="centered hidden">
```

The form no longer appears because of the addition of the hidden class.

- 3 Examine `lab09a-ex04.css`. Notice the two classes `hidden` and `visible`.

- 4 Add the following code to `lab09a-ex04.js` and test.

```
setTimeout( function () {
  let aside = document.querySelector('aside');
  aside.className = 'visible';
}, 3000);
```

*After a three second delay, this sets the class for the `<aside>` to `visible`. Unfortunately, it also removes the `centered` class since setting `className` **replaces** any existing classes.*

- 5 Modify the code as follows and test.

```
setTimeout( function () {
  let aside = document.querySelector('aside');
  aside.classList.remove('hidden');
  aside.classList.add('visible');
}, 3000);
```

Using `classList` allows you to specify multiple classes (`className` only allows one).

- 6 As an alternate approach, simplify the code using the `toggle` method as follows and then test.

```
setTimeout( function () {
  let aside = document.querySelector('aside');
  aside.classList.toggle('hidden');
}, 3000);
```

Each call to `toggle()` either adds or removes the class from the element.

TEST YOUR KNOWLEDGE #2

In Chapter 8, in the fourth Test Your Knowledge, you used the `document.write()` method to output markup as strings. In this exercise, you will use DOM methods to dynamically add similarly structured content instead so that your output looks similar to that shown in Figure 9.2.

- 1 Examine `lab09a-test02.html`. It provides the sample markup for a single photo. You will replace that markup with JavaScript in the following steps. For now, either comment out the markup or cut it and paste into a temporary file for referencing later.
- 2 Modify `lab09a-test02.js` in your editor. You will need to select the `<section>` element that has an `id=parent`. Loop through the `photos` array and create a `<figure>` element that will get appended to the parent element. You may want to examine `photos.json` again to reacquaint yourself with its structure.
- 3 Within that loop, you will need to dynamically create the `<img>` element using the appropriate DOM methods. Populate its `alt` attribute using the `title` property from each photo.
- 4 After the image is created, you will need to create the `<figcaption>` element. This element will contain the following child elements which will also be dynamically generated from the photo data: `<h2>`, `<p>`, and `<span>` elements. For the `<span>` elements, you will need to loop through the `colors` array inside each photo object and set the `backgroundColor` property of the `<span>` to the hex value. Set the `title` attribute of each `span` to the hex value. Append the `<figcaption>` to the `<figure>`.
- 5 Be sure to append the created elements to the appropriate parent. Because of all the nested elements, this code can get pretty messy. Be sure to make use of your own functions to keep your code more manageable (and expressive).

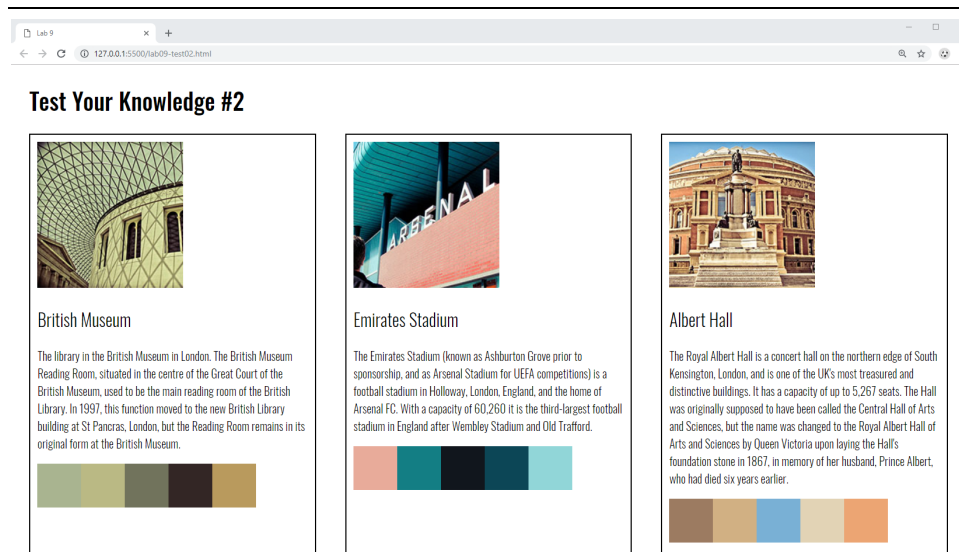


Figure 9.2 – Finished Test Your Knowledge #2

EVENT HANDLING

One of the most common uses of JavaScript is to use it to respond to different user-initiated actions. These are generally referred to as **events** and the JavaScript that responds to these events are called **event handlers**. As the textbook indicates, the preferred approach is to use event listeners.

EXERCISE 9a.5 — SIMPLE EVENT HANDLING

- 1 Examine `lab09a-ex05.html` in your browser. Open `lab09a-ex05.js` then add the following and test.

```
function simpleHandler(event) {
    alert("button was clicked");
}
const btn = document.getElementById("firstButton");
btn.addEventListener("click", simpleHandler);
```

This defines an event handler function and assigns it to the click event of the first button. It is quite common to instead use an anonymous function for the event handler, as shown in the next steps.

- 2 Modify the following code and test.

```
const btn = document.getElementById("firstButton");
btn.addEventListener("click", function () {
    alert("a different approach but same result");
});
```

While this takes a bit of getting used to, it is an extremely common way of doing JavaScript event coding.

- 3 Modify the code by using arrow function syntax and test

```
const btn = document.getElementById("firstButton");
btn.addEventListener("click", () => {
    alert("this time using arrow syntax");
});
```

- 4 What if we wanted the same event handler for all three buttons? Comment out the code added in previous step, then add the following code and test.

```
// illustrates one event handler for multiple events
const buttons = document.querySelectorAll(".card button");
for (let btn of buttons) {
    btn.addEventListener("click", () => {
        alert("now just one handler");
    });
}
```

Notice how this selects a collection of elements and then loops through them to assign event handlers. But what if we wanted to know more about the element that generated the event?

5 Modify the function code as follows and test.

```
btn.addEventListener('click', () => {  
    alert(btn.id);  
});
```

Because of JavaScript closure, the callback function has access to `btn` because it was in scope when the function was defined.

6 Modify the function code as follows and test.

```
btn.addEventListener('click', (e) => {  
    alert(e.target.id);  
});
```

Notice the use of the event object parameter (the variable named `e`). It provides the callback function with information about the generating event. In this example `e.target` is the button element that was clicked.

7 Modify the code as follows and test.

```
for (let btn of buttons) {  
    btn.addEventListener('click', clickHandler);  
}  
function clickHandler (e) {  
    let para = e.target.previousElementSibling;  
    alert(para.textContent);  
}
```

The event object is essential if the event handler is not an anonymous function, and we can no longer rely on scope.

8 Add the following code and test by moving mouse in and out of the images.

```
// illustrates the assigning of multiple handlers to the same object  
const images = document.querySelectorAll(".card img");  
for (let img of images) {  
    img.addEventListener('mouseover', e => {  
        e.target.classList.add('sepia');  
    });  
    img.addEventListener('mouseout', e => {  
        e.target.classList.remove('sepia');  
    });  
}
```

This applies a CSS filter dynamically to the card image when you mouse over the image, and removes the filter when the mouse is moved out of the image. Recall from Exercise 9a.4, that you can use `classList.toggle()` instead. Notice also how the code is more concise due to the use of arrow syntax.

EXERCISE 9a.6 — RESPONDING TO LOAD EVENTS

- 1 In `lab09a-ex06.html` (which is the same file as the finished version of the previous exercise), move the `<script src="js/lab09a-ex06.js"></script>` from the end of the document to within the `<head>` element instead.

- 2 Test. Display the JavaScript console in your browser

Depending on your browser and version, you may or may not get a console error. The code will likely no longer work. Why?

- 3 The reason it no longer works is that the script is now executed **before** the DOM is fully loaded. To make the script execute **after** the DOM is loaded, we must make use of the page load event (called `DOMContentLoaded`). Try this yourself by moving the event handling code inside of a listener for the `DOMContentLoaded` event as follows (some code omitted) and then test:

```
// don't set up event handlers until DOM is loaded
document.addEventListener('DOMContentLoaded', function () {

    // illustrates one event handler for multiple events
    const buttons = document.querySelectorAll(".card button");
    for (let btn of buttons) {
        btn.addEventListener('click', function (e) {
            let para = e.target.previousElementSibling;
            alert(para.textContent);
        });
    }
    ...
});
```

Almost all JavaScript code you write from now on, will happen within the `DOMContentLoaded` handler.

EXERCISE 9a.7 — MORE EVENT HANDLING

- 1 Examine `lab09a-ex07.html` in the browser.

In this exercise, you will be dynamically updating the calculations based on user input.

- 2 Begin by adding a `DOMContentLoaded` handler to `lab09a-ex07.js`.

```
document.addEventListener('DOMContentLoaded', () => {
  // this will be replaced by code in the next steps
});
```

- 3 Add the following code:

```
document.addEventListener('DOMContentLoaded', () => {
  // references to input elements
  const numQuantity = document.querySelector("#numQuantity");
  const numPrice = document.querySelector("#numPrice");
  const numTaxRate = document.querySelector("#numTax");

  // implement the inc/dec quantity functionality
  const btnInc = document.querySelector("#increaseBtn");
  btnInc.addEventListener('click', () => {
    if (numQuantity.value < 5) numQuantity.value++;
    calcTotals();
  });
  const btnDec = document.querySelector("#decreaseBtn");
  btnDec.addEventListener('click', () => {
    if (numQuantity.value > 0) numQuantity.value--;
    calcTotals();
  });

  // recalculate when input elements change
  numPrice.addEventListener('input', calcTotals );
  numTaxRate.addEventListener('input', calcTotals );
  numQuantity.addEventListener('input', calcTotals );
});
```

- 4 Implement the `calcTotals` function as follows (inside the `DOMContentLoaded` handler) and then test.

```
// re-calculate and re-display the totals
function calcTotals() {
  // get references to relevant form elements
  const divSubtotal = document.querySelector("#subtotal");
  const divTax = document.querySelector("#taxAmount");
  const divGrandTotal = document.querySelector("#grandTotal");
  // do the calculations
  let subtotal = numQuantity.value * numPrice.value;
  let tax = subtotal * (numTaxRate.value/100);
  let grandtotal = subtotal + tax;
  // display the new numbers
  divTax.textContent = "$" + tax.toFixed(2);
  divSubtotal.textContent = "$" + subtotal.toFixed(2);
  divGrandTotal.textContent = "$" + grandtotal.toFixed(2);
}
```

It should work! It should look similar to that shown in Figure 9.3.

- 5 Add the following code to implement the accordion (an element that collapses and opens) functionality.

```
// implement the accordion functionality
const header =
  document.querySelector("div.collapsible-header");
header.addEventListener('click', e => {
  const content =
    document.querySelector("div.collapsible-content");
  const arrow = header.querySelector('.arrow');

  // Toggle the expanded class
  content.classList.toggle('expanded');
  arrow.classList.toggle('expanded');
});
```

The actual work hiding/showing the content is achieved via CSS; if you examine the CSS, you will see the non-expanded version of the content area has a height of 0 and no overflow, thereby hiding the content.

More DOM Manipulation

This example will demonstrate JavaScript's dynamic capabilities.

Calculate

Price: 2500.00

Quantity: - 2 +

Tax Rate (%): 12

Subtotal: \$5000.00

Tax: \$600.00

Grand Total: \$5600.00

Figure 9.3 – Finished Exercise 9a.7

The next exercise is going to illustrate a common use case in JavaScript: given some data in an array, dynamically generate a list (or a table) to display that data, complete with event handling. As you will see in this exercise, the code to do this in regular DOM JavaScript (sometimes referred to as or vanilla JavaScript) can get quite long-winded and be a bit tedious to write.

EXERCISE 9a.8 — WORKING WITH LISTS

- 1 Examine `lab09a-ex08.html` in your browser and then in your code editor. Notice the sample `<li>` element, which you will replace with your own code.
- 2 Add the following helper functions to `lab09a-ex08.js`.

```
// helper function to create list <span> element
function createSpan(content, className) {
  const span = document.createElement("span");
  span.appendChild(content);
  if (className) span.className = className;
  return span;
}

// helper function to create <button> element
function createButton(className) {
  const btn = document.createElement("button");
  const icon = document.createElement("i");
  icon.classList.add("fa-solid");
  if (className) icon.classList.add(className);
  btn.appendChild(icon);
  return btn;
}
```

- 3 Add the following code inside the `DOMContentLoaded` handler:

```
document.addEventListener("DOMContentLoaded", () => {
  const list = document.querySelector("#list");
  /* for each stock in provided array, add a list item */
  for (let stck of stocks) {
    const li = document.createElement("li");
    li.className = "grid";

    // add stock symbol to <li>
    li.appendChild(
      createSpan(document.createTextNode(stck.symbol), "");

    // add stock name to <li>
    const a = document.createElement("a");
    a.textContent = stck.name;
    a.href = "#";
    li.appendChild( createSpan(a, "bold") );

    // add stock address and sector to <li>
    li.appendChild(
      createSpan(document.createTextNode(stck.address), "");
    li.appendChild(
      createSpan(document.createTextNode(stck.sector), "");

    // add two buttons to <li>
    const btnView = createButton("fa-magnifying-glass");
    const btnHeart = createButton("fa-heart");
    li.appendChild(btnView);
    li.appendChild(btnHeart);

    // add li to the <ul> list
    list.appendChild(li);
  }
}
```

- 4 Remove (or comment out) the sample `<li>` element in `lab09a-ex08.html`.
- 5 Test.

The result should look similar to that shown in Figure 9.4.

List

MMM	3M Company	St. Paul, Minnesota	Industrials	Q	♥
ABT	Abbott Laboratories	North Chicago, Illinois	Health Care	Q	♥
ACN	Accenture plc	Dublin, Ireland	Information Technology	Q	♥
ATVI	Activision Blizzard	Santa Monica, California	Information Technology	Q	♥
AYI	Acuity Brands Inc	Atlanta, Georgia	Industrials	Q	♥

Figure 9.4 – Exercise 9a.12 after step 5.

- 6 Add the following event handling code (some code omitted). Test.

```
const btnHeart = createButton("fa-heart");

btnView.addEventListener('click', e => {
  alert('button was clicked, but on which row?');
});
```

- 7 Modify the event handler as follows and then test.

```
btnView.addEventListener('click', e => {
  alert('button was clicked symbol=' + stck.symbol);
});
```

This works because of JavaScript scope (this code is within the loop which defines `stck`).

- 8 Modify the event code as follows.

```
btnView.addEventListener('click', viewHandler);
```

Unfortunately, the handler function won't have access to `stck`, so you will need to "inject" the relevant stock information "into" the `<button>` element.

- 9 Add the following line.

```
btnView.dataset.symbol = stck.symbol;
btnView.addEventListener('click', viewHandler);
```

Notice the use of the `dataset` property, which is an interface for accessing `data-*` attributes. The `data-*` attribute is a mechanism within HTML5 for adding custom information to an element. Multiple attributes of the form `data-AAAA` can be added to an element, where `AAAA` is any valid string. This code will add an attribute-value pair, such as `data-symbol="ABC"`, to each view button.

- 10 Add the following event handling function and test.

```
function viewHandler(e) {
  let symbol;
  // the user can click the <button> or the <i> element
  if (e.target.nodeName.toLowerCase() == 'button')
    symbol = e.target.dataset.symbol;
  else
    symbol = e.target.parentNode.dataset.symbol;

  // find stock with matching symbol
  let foundStock;
  for (let s of stocks) {
    if (s.symbol == symbol) foundStock = s;
  }
  if (foundStock)
    alert('button was clicked stock=' + foundStock.name);
}
```

It should work!

EXERCISE 9a.9—EVENT PROPAGATION

- 1 Examine `lab09a-ex09.html` in your browser. Open `lab09a-ex09.js` then add the following (note that the code is already wrapped in a `DOMContentLoaded` handler).

```
document.addEventListener("DOMContentLoaded", function() {

  const containers = document.querySelectorAll('.container');
  for (cont of containers) {
    cont.addEventListener('click', e => {
      console.log(`${e.target.nodeName} was clicked -
        Propagated to ${e.currentTarget.nodeName}`);
    });
  }
})
```

This adds this event handler to every element in the document. See Figure 9.13 in the book for an explanation of the difference between `e.target` and `e.currentTarget`.

- 2 Test by clicking on one of the `<span>` elements and examine the output in the console. Experiment by clicking on other elements. You may want to use the clear console button.

Notice how the click event propagates up the DOM tree to the different ancestors of the element that was clicked.

- 3 Add the following code (within the `DOMContentLoaded` handler) then test by clicking one of the `<span>` elements.

```
const spans = document.querySelectorAll('span');
for (s of spans) {
  s.addEventListener('click', function (e) {
    console.log('special extra handler for span');
    e.stopPropagation();
  });
}
```

This stops the event from propagating up the ancestor chain.

- 4 Modify the code as follows.

```
document.addEventListener("DOMContentLoaded", function() {

  const containers = document.querySelectorAll('.container');
  for (cont of containers) {
    cont.addEventListener('click', function(e) {
      console.log(`${e.target.nodeName} was clicked -
        Propagated to ${e.currentTarget.nodeName}`);
    }, { capture: true });
  }
})
```

This changes the way this event “moves”. In the default mode, events propagate from the closest object “up” to its ancestors. By turning on capture, events move in the opposite direction, from the furthest ancestor “down” to the closest object.

- 5 Add the following code (within the `DOMContentLoaded` handler) then test by clicking one of the `<span>` elements.

```
const section = document.querySelector('section');
section.addEventListener('click', function (e) {
  e.stopPropagation();
}, {capture: true});
```

Now the event “stops” when it reaches the `<section>` element and never makes it to the “lower” event handler for the `<span>` elements.

EXERCISE 9a.10—EVENT DELEGATION

- 1 Examine `js/stocks.js`. Recall from Chapter 8 that JSON format is often used to transport data for JavaScript consumption. Here we will simply be reading the JSON data from a file. Next chapter we will be receiving it from an external web API, which is more indicative of real-world usage.
- 2 Add the following code to `lab09a-ex10.js` and then test by viewing `lab09a-ex10.html` in the browser.

```
const stocks = JSON.parse(stockData);  
console.dir(stocks);
```

You should be able to see `stocks` is an array of objects, each one representing a single stock.

- 3 Add the following and test.

```
document.addEventListener("DOMContentLoaded", () => {  
  const main = document.querySelector('main');  
  // loop through stock data and output each one in a div  
  for (let stock of stocks) {  
    // create img and div container for img  
    let img = document.createElement('img');  
    img.setAttribute('src',  
      `images/logos/${stock.symbol}.svg`);  
    img.setAttribute('title', stock.symbol);  
    let div = document.createElement('div');  
    div.setAttribute('class', 'item');  
    // add img to div and div to <main>  
    div.appendChild(img);  
    main.appendChild(div);  
  }  
});
```

You should now see about 70 company logos.

- 4 When the user clicks on a logo, we want to display more information about that company. Instead of looping through each image and assigning event handlers to each, we will instead use event delegation. Add the following after the `for` loop from previous step:

```
// use delegation to handle click events for all the images
document.querySelector('main')
  .addEventListener('click', function (e) {
    // verify user has clicked on image within <main>
    if (e.target && e.target.nodeName.toLowerCase() == 'img') {
      populateAside(e);
    }
  });
```

For historical reasons, the `nodeName` of an element is always upper case.

- 5 Now write the action for the click. In our case, you are going to populate the table in the `<aside>` element with the relevant company data contained in our `stocks` array. Add the following and test.

```
function populateAside(e) {
  // determine the clicked symbol name from clicked image
  let clickedSymbolName = e.target.getAttribute('title');

  // search through stocks array looking for symbol that matches
  const foundSymbol = stocks.find(function(element) {
    return element.symbol === clickedSymbolName;
  });

  // display aside (hidden initially)
  let aside = document.querySelector('aside');
  aside.style.display = 'block';

  let logo = document.querySelector('#logo img');
  let symbol = document.querySelector('#symbol');
  let name = document.querySelector('#name');
  let sector = document.querySelector('#sector');
  let sub = document.querySelector('#sub');

  // populate table with data
  logo.setAttribute('src',
    `images/logos/${foundSymbol.symbol}.svg`);
  symbol.textContent = foundSymbol.symbol;
  name.textContent = foundSymbol.name;
  sector.textContent = foundSymbol.sector;
  sub.textContent = foundSymbol.subIndustry;
}
```

The `find()` method of the array object will be covered in Chapter 10.

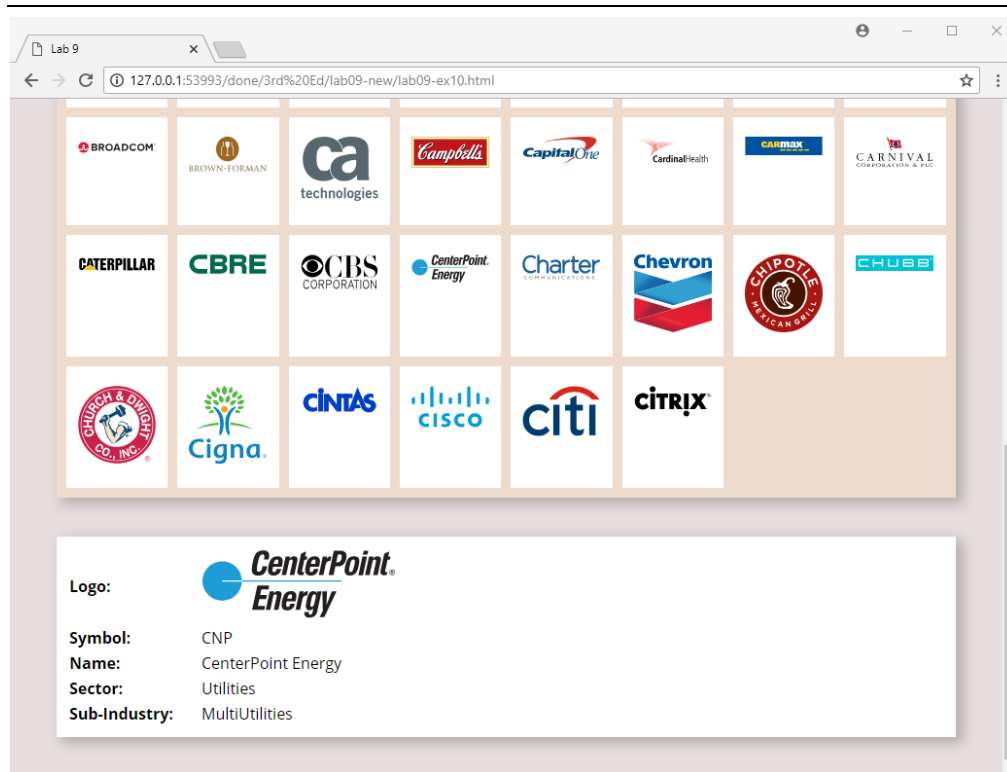


Figure 9.5 – Finished Exercise 9a.10

Event delegation is an essential technique that you must master. But there are a variety of approaches for implementing it. The next example illustrates a common alternative approach to using event delegation. It uses a conditional check on the class name rather than the tag name. It also illustrates how to respond to `<select>` element events and to mouse events.

EXERCISE 9a.11 — ALTERNATE EVENT DELEGATION TECHNIQUES

- 1 Examine `lab09a-ex11.html` in your browser and then examine in text editor.
Notice that there are two separate containers for the color boxes. We want a single event handler to work with both box containers. This will require testing against a class name rather than a tag name.
- 2 Examine `lab09a-ex11.js`. Notice it already has code for some of the functionality.
- 3 Add the following code to `lab09a-ex11.js` in the indicated space and test.

```
const selector = document.querySelector(".controls select");
// handle new selection of the <select>
selector.addEventListener("change", e => {
  if (selector.value !== 0) {
    createBox(selector.value);
  }
});
```


- 4 Add the following code and test.

```
const container = document.querySelector("main.container");
// Single event listener using event delegation
container.addEventListener('click', e => {
  // check if clicked element has the box class
  if (e.target.classList.contains('box')) {
    // get its color from the data-color attribute
    const color = e.target.dataset.color;
    // let user know in event log area
    logEvent("CLICK", ` ${color} box`);
    // Add some visual feedback
    e.target.style.transform = 'scale(0.95)';
    setTimeout(() => {
      e.target.style.transform = '';
    }, 150);
  }
});
```

Notice that we are listening to the parent element of both color containers.

- 5 Add the following code and test. The result should look similar to Figure 9.6.

```
container.addEventListener('mouseover', e => {
  // check if clicked element has the box class
  if (e.target.classList.contains('box')) {
    const color = e.target.dataset.color;
    logEvent("HOVER", ` ${color} box`);
  }
});
```

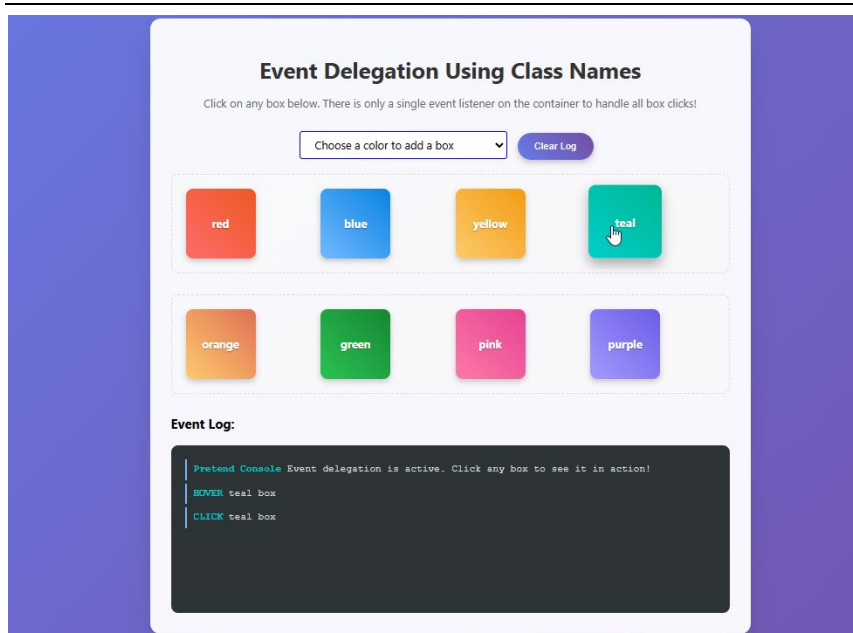


Figure 9.6 – Finished Exercise 9a.11

TEST YOUR KNOWLEDGE #3

Examine `lab09a-test03.html`, view in browser, and then open `lab09a-test03.js` in your editor. You will be using JavaScript to implement a shopping cart. Notice the two commented areas in the markup.

- 1 Create a constructor function to represent a single cart item. It should contain the following properties: `id`, `name`, `price`, and `quantity`.
- 2 Add one or more global variables that will be used to implement the shopping cart state. You will need a global array variable that will store the items currently in the shopping cart. You may also decide to have global variables for the total number of items in the cart and its total value amount. Alternately, you could generate those two numbers dynamically.
- 3 You will need to use a loop to add event handlers for each of the Add to Cart buttons. To make your code more understandable, call a function that will add an item to the cart.
- 4 Implement the add to cart function. You can retrieve the information about the clicked movie via the `dataset` property. The HTML structure of a cart item can be seen in the commented example in the markup. If an item is already in the cart, increase its quantity.
- 5 When a cart item is added, the cart item count (in red circle) and the total cart value (in green) should change. The final result should look similar to that shown in Figure 9.7.

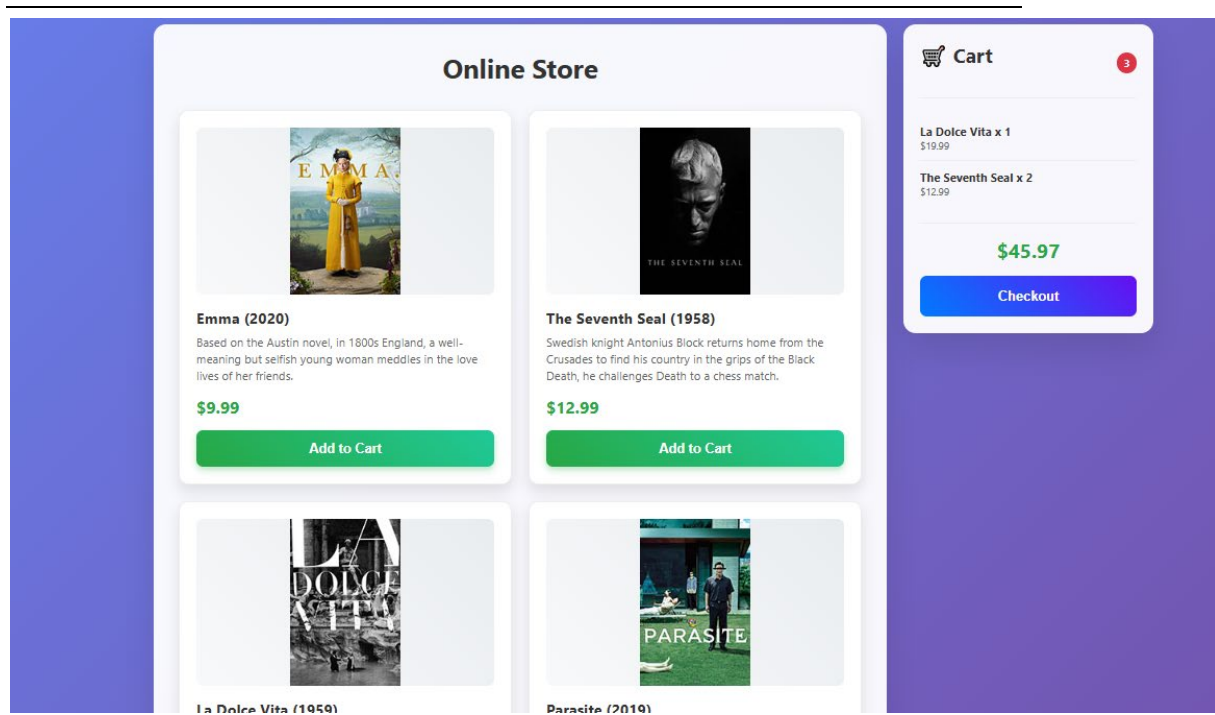


Figure 9.7 – Finished Test Your Knowledge #3

EXERCISE 9a.12 — WORKING WITH FORMS

- 1 Examine `lab09a-ex12.html` in your browser.
- 2 Add the following function to `lab09a-ex12.js`:

```
function setBackground(e) {
  if (e.type == "focus") {
    e.target.style.backgroundColor = "#FFE393";
  }
  else if (e.type == "blur") {
    e.target.style.backgroundColor = "white";
  }
}
```

This function is going to get called every time the focus (user enters a control) or blur (user leaves a control) events are triggered in one of our form's input elements.

- 3 Add the following code:

```
document.addEventListener("DOMContentLoaded", () => {
  const cssSelector = "input[type=text],input[type=password]";
  const fields = document.querySelectorAll(cssSelector);

  for (let field of fields) {
    field.addEventListener("focus", setBackground);
    field.addEventListener("blur", setBackground);
  }
});
```

The anonymous function on the load event assigns the `setBackground()` to the blur and focus events of the relevant `<input>` elements.

- 4 Test in browser. Move between the different elements with the tab key or mouse.

EXERCISE 9a.13 — FORM VALIDATION

- 1 Examine `lab09a-ex13.html` in your browser. It is the same as the ending point of the previous exercise.
- 2 Add the following code to `lab09a-ex13.js`.

```
for (let field of fields) {
  field.addEventListener("focus", setBackground);
  field.addEventListener("blur", setBackground);
}

/* code goes here */

// when user submits form, check for empty fields
document.querySelector("#sampleForm").
  addEventListener("submit", checkForEmptyFields);

// ensures form fields are not empty
function checkForEmptyFields(e) {
  // loop thru the input elements looking for empty values
  const errorList = [];
  for (let field of fields) {
```

```

        if (field.value == null || field.value == "") {
            // since a field is empty prevent the form submission
            e.preventDefault();
            errorList.push(field);
        }
    }

    // now display the error message if any
    let msg = "The following fields can't be empty: ";
    if (errorList.length > 0) {
        for (let errorField of errorList) {
            msg += errorField.id + ", ";
        }
        alert(msg);
    }
}

```

- 3 Test in browser. Experiment by filling in some fields, leaving others blank. Also try putting just spaces in some fields.

- 4 Modify the `checkforEmptyFields()` function by adding the following:

```

const errorList = [];
for (let field of fields) {
    if (field.value == null || field.value == "") {
        // since a field is empty prevent the form submission
        e.preventDefault();
        errorList.push(field);
    }
}

// hide the error message element in case it is visible
const errorArea = document.getElementById("errors");
errorArea.className = "hidden";

// now display the error message if any
let msg = "The following fields can't be empty: ";
if (errorList.length > 0) {
    for (let errorField of errorList) {
        msg += errorField.id + ", ";
    }
    // replace the alert; for this simple example, use innerHtml
    errorArea.innerHTML = "<p>" + msg + "</p>";
    errorArea.className = "visible";
}

```

Instead of displaying the error message inside an alert box this places it within a `<div>` element. The result should look similar to that shown in Figure 9.8.

Registration

User Name

✕

Phone

✕

Email

✕

Password

✕

Register

The following fields can't be empty: username, phone, email, pass,

Figure 9.8 – Finished Exercise 9a.13

TEST YOUR KNOWLEDGE #4

Examine `lab09a-test04.html`, view in browser, and then open `lab09a-test04.js` in your editor. Modify the JavaScript file to implement the following functionality.

- 1 Add an event handler for the Add Task button. This handler needs to make use of `e.preventDefault()` so that it doesn't try to submit the form.
- 2 It should also remove the `hidden` class from the `div` element, and add the `visible` class instead.
- 3 Create a constructor function (see Lab 8) to represent a task object. You will need to define two fields in this object: the task details and the task date. The easiest value for this identifier is the value of the `Date.now()` function.
- 4 Define an array of task objects; every time the Add task button is clicked, add a task object to this array.
- 5 Add a new table row that contains the task date and details to the `taskTable`.
- 6 Clear the two input values (set their `value` property to `""`).

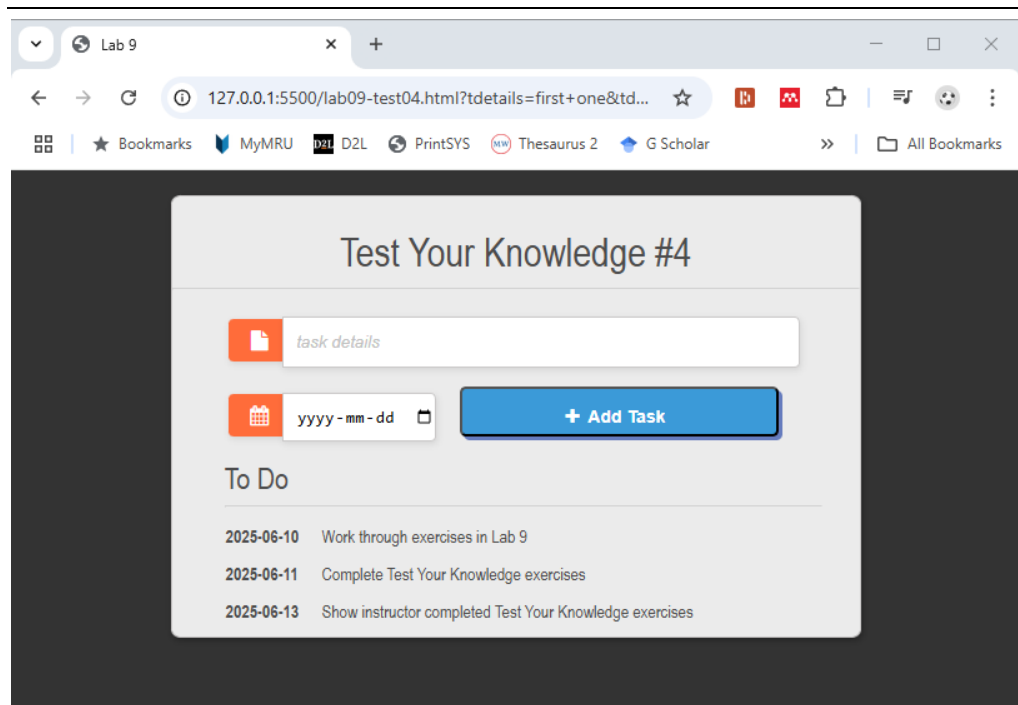


Figure 9.9 – Finished Test Your Knowledge #4

An alternative approach to modifying the DOM makes use of the new `<template>` element added in HTML5. This element “defines” HTML content structure that is not immediately rendered by the browser (i.e., it is ignored by the browser), but is populated later using JavaScript. This approach is very helpful when the markup one is creating is quite complicated, for instance, with many CSS classes to add. The next example illustrates such a case. It uses Tailwind CSS, which is an extremely popular CSS framework and populates the markup with dozens or even hundreds of CSS class references. For such an example, the template method is almost a necessity.

EXERCISE 9a.14 — WORKING WITH TEMPLATES

- 1 Examine `paintings.js`. You will be using this data in this exercise.
- 2 Examine `lab09a-ex14.html` in your browser and then in your code editor. Notice the sample painting card. With all of the Tailwind CSS classes, this would be unpleasant and tedious work to implement dynamically via the DOM techniques covered so far in this lab. Instead, you will use the `<template>` alternative.
- 3 Modify the markup by wrapping the card markup in a `<template>` tag.

```
<template id="sampleCard">
  <div class="bg-white rounded-lg shadow-lg ...">
    <img class="w-full h-48 object-cover" src="" alt="">
    ...
  </div>
</template>
```

- 4 Add the following to `lab09a-ex14.js` and test

```
document.addEventListener("DOMContentLoaded", () => {

  const template = document.querySelector('#sampleCard');
  const parent = document.querySelector('#parent');

  // loop thru each painting
  for (let pnt of paintings) {
    // here is the magic: making a clone of the template
    const clone = template.content.cloneNode(true);

    // and now we insert content into the clone
    const img = clone.querySelector("img");
    img.setAttribute("src",
      `images/paintings/${pnt.imageFileName}`);
    img.setAttribute("alt", `${pnt.title}`);

    const h2 = clone.querySelector("h2");
    h2.textContent = pnt.title;
    const h3 = clone.querySelector("h3");
    h3.textContent = pnt.firstName + ' ' + pnt.lastName;
    const h4 = clone.querySelector("h4");
    h4.textContent =
      `${pnt.gallery.name} (${pnt.yearOfWork})`;
    const p = clone.querySelector("p");
    p.textContent = pnt.details.excerpt;

    const btn = clone.querySelector("button");
    btn.dataset.painting = pnt.paintingID;
    btn.addEventListener('click', e => {
      alert(`You clicked on painting
        ${e.target.dataset.painting}`);
    });

    // and then add the clone to the document
    parent.appendChild(clone);
  }
});
```

TEST YOUR KNOWLEDGE #5

You will expand your completed `lab09a-test04` and add the following functionality.

- 1 In the `aside` element is a Remove All button. When a task is added, remove the `hidden` class from the `aside` element, and add the `visible` class instead.
- 2 Add an event handler for this Remove button that empties the task array and removes the rows from the table.
- 3 Add a new column that contains a Delete button. Each Delete button will need to contain a unique identifier for the task using the `dataset` property (see Exercise 9a.8). The event handler for this button will use the value of the `dataset` to delete the relevant task from the task array, and then redraw the task table using the modified array.

The result should look similar to that shown in Figure 9.10.

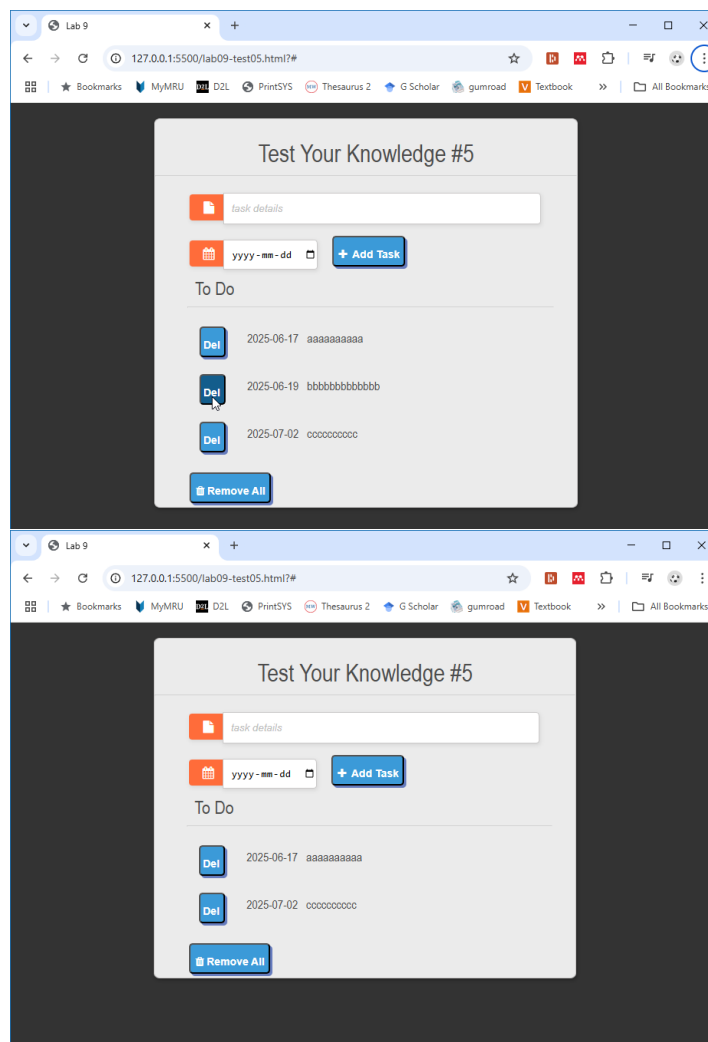


Figure 9.10 – Finished Test Your Knowledge #5